Project Falcon: Database Scaling Design

Halcyon Labs Data Infrastructure · Design document DI-FALCON-002 · Approved March 2025

1. Background

Project Falcon is the data-layer redesign for Halcyon's merchant platform. Before Falcon, all merchant data lived in a single PostgreSQL primary with two read replicas. By late 2024 the primary was handling 31,000 writes per second at peak and storage had grown to 14 TB. Vertical scaling had reached its limit: the largest available instance was already in use, and vacuum operations were causing latency spikes every afternoon.

This document explains the fundamentals Falcon relies on — replication, sharding, consistent hashing and the CAP theorem — and records the specific decisions the team made.

2. Replication

Replication keeps copies of the same data on multiple machines. It improves read throughput, because reads can be spread across replicas, and availability, because a replica can take over if the primary fails. The cost is that replicas may lag behind the primary.

2.1 Replication models

- **Single-leader (primary–replica):** all writes go to one node, which streams changes to followers. Simple and widely used. PostgreSQL streaming replication works this way.
- **Multi-leader:** several nodes accept writes and sync with each other. Useful across data centres but requires conflict resolution.
- **Leaderless:** clients write to several replicas directly and read from several, using quorums. Cassandra and DynamoDB follow this model.

2.2 Falcon's replication decisions

Falcon uses single-leader replication within each shard, with a **replication factor of 3**: one primary and two replicas spread across three availability zones in the Mumbai region. One replica is synchronous and one is asynchronous, so a committed write always exists on at least two machines. An alert fires when asynchronous replication lag exceeds **800 milliseconds** for more than one minute. Automatic failover is handled by Patroni, and the target recovery time objective (RTO) for a shard primary failure is **90 seconds**.

3. Sharding

Sharding (horizontal partitioning) splits a dataset across multiple databases so that each holds only part of the data. Unlike replication, which copies everything, sharding divides the data, so it scales both storage and write throughput. The hardest decision is the **shard key**: the field used to decide which shard a row belongs to.

3.1 Sharding strategies

Strategy	How it works	Strength	Weakness	Used in Falcon?
Range-based	Rows assigned by key ranges, e.g. A–F, G–M	Efficient range queries	Hot spots on popular ranges	No

Strategy	How it works	Strength	Weakness	Used in Falcon?
Hash-based (modulo)	shard = hash(key) mod N	Even distribution	Adding a shard remaps almost all keys	No
Consistent hashing	Keys and nodes placed on a hash ring	Adding a node moves only ~1/N of keys	More complex routing	Yes
Directory-based	Lookup table maps key to shard	Fully flexible	Lookup service is a bottleneck	Only for 12 enterprise merchants
Geo-based	Rows placed by user region	Data residency, low latency	Uneven load across regions	Planned for 2026

3.2 Falcon's sharding decisions

Falcon shards on **merchant_id**. Nearly every query in the merchant platform is scoped to one merchant, so this key keeps each merchant's orders, products and settlements on the same shard and avoids cross-shard joins. The team considered order_id as the shard key but rejected it because merchant dashboards would then need to query every shard.

Data is divided into **64 logical shards** placed on **8 physical database clusters**, eight logical shards per cluster. Starting with more logical shards than machines means the team can later move a logical shard to a new cluster without re-splitting data.

4. Consistent hashing in Falcon

In consistent hashing, both nodes and keys are hashed onto a circular space (the ring). A key belongs to the first node found by moving clockwise from the key's position. When a node is added, it only takes over keys from its immediate neighbour, so roughly 1/N of keys move instead of nearly all of them, as happens with modulo hashing.

With only a few nodes, the ring is uneven: one node might own a much larger arc than another. The fix is **virtual nodes**: each physical node is placed on the ring many times. Falcon uses **150 virtual nodes per physical cluster** and the **xxHash64** hash function. The team tested 50, 150 and 400 virtual nodes; at 150 the largest cluster held only 4 percent more keys than the average, while 400 gave little extra balance and made the routing table noticeably larger.

Routing is performed by a small service called **Perch**, which keeps the ring in memory and is embedded as a library in every application. Perch reloads the ring from etcd whenever it changes.

5. The CAP theorem and Falcon's choices

The CAP theorem says that when a network partition happens, a distributed system must choose between **consistency** (every read sees the latest write) and **availability** (every request gets a response). Partitions cannot be avoided in practice, so the real choice is between CP and AP behaviour during a partition. The PACELC extension adds that even without a partition, systems trade latency against consistency.

- **Settlement ledger — CP.** A merchant's balance must never be wrong. During a partition, writes to the ledger are rejected rather than risk divergence.

- **Product catalogue — AP.** Showing a slightly stale product description is acceptable; showing an error page is not. Catalogue reads continue from any replica during a partition.
- **Merchant activity feed — AP** with eventual consistency, stored in Cassandra with quorum settings of W=2 and R=1 on a replication factor of 3.

6. Migration plan and results

Migration ran in three phases over 11 weeks. First, the team enabled dual writes from the old database to the new shards. Second, a backfill job copied historical data at a capped rate of 20,000 rows per second to avoid hurting production. Third, reads were shifted shard by shard behind a feature flag, starting with 1 percent of merchants.

After cut-over in June 2025, peak write capacity rose to roughly 120,000 writes per second, p99 write latency fell from 340 milliseconds to 45 milliseconds, and the afternoon vacuum spikes disappeared. The project lead was Farah Iqbal, Principal Engineer, Data Infrastructure.

7. Known risks

- A single very large merchant can still create a hot shard. Merchants above 5 percent of total traffic are moved to the directory-based path.
- Cross-merchant analytics cannot run on Falcon shards and must use the nightly data-warehouse export.
- Resharding still requires a manual runbook; automating it is planned for Falcon Phase 2.